

NTI - Digital Design using FPGA

Contents

1	Directory: Lab_1_Q1	3
1.1	Architecture: Dataflow Modeling of a Full Adder	3
2	Directory: Full_Adder_B	3
2.1	Architecture: Behavioral Abstraction	3
3	Directory: Full_Adder_g	3
3.1	Architecture: Gate-Level Structural Design	3
4	Directory: Counter_behavioral	4
4.1	Architecture: Synchronous Sequential Logic	4
5	Directory: Counter_gate	4
5.1	Architecture: Primitive Sequential State Control	4
6	Directory: counter	5
6.1	Architecture: Pure Structural Instantiation	5
7	Directory: Fsm	5
7.1	Architecture: Finite State Machine Control Logic	5
8	Directory: Lab_5_alu	5
8.1	Architecture: Parameterizable Datapath Processing	5
9	Directory: Lab_3_sheft	6
9.1	Architecture: Serial-to-Parallel Interfacing	6
10	Directory: Lab_7_register	6
10.1	Architecture: Datapath Storage Elements	6

11 Directory: Lab_8_memory	6
11.1 Architecture: Static RAM (SRAM) Modeling	6
12 Directory: Lab_9_counter	7
12.1 Architecture: Program Counter (PC) Mechanics	7
13 Directory: Lab_6_controller	7
13.1 Architecture: CPU Instruction Decoder and FSM	7
14 Directory: Lab_4_Top_module	7
14.1 Architecture: Advanced SoC Integration & Data Routing	7
15 Directory: Final project	8
15.1 Graduation Project: FPGA-Based Multi-Antenna Controller for Microwave Head Imaging	8
15.1.1 The Problem Statement	8
15.1.2 Our Proposed Solution: FPGA Hardware Control	8
15.1.3 System Architecture & Module Integration	9
15.1.4 The FSM: Orchestrating the Cycle	9
15.1.5 Verification and Testing Strategy	10

1 Directory: Lab_1_Q1

1.1 Architecture: Dataflow Modeling of a Full Adder

This directory contains the foundational implementation of a 1-bit Full Adder utilizing the Dataflow modeling paradigm in Verilog. In digital logic design, dataflow modeling abstracts the circuit into continuous boolean equations rather than discrete physical gates. This is a critical abstraction layer that allows engineers to design based on boolean algebra rather than transistor-level physics.

The architecture relies heavily on continuous assignment statements to model the combinational logic. The logical equations for a full adder are fundamentally structured around bit-wise operations. The sum is generated using cascaded XOR operations, which effectively determine the odd/even parity of the input bits. Meanwhile, the carry-out logic determines if a carry is generated locally (via an AND operation between the main inputs) or if an incoming carry is propagated through the XOR partial sum.

By utilizing intermediate wire declarations, the logic is segmented into physical conceptual nodes. This approach is highly efficient for synthesis tools (like Xilinx Vivado or Intel Quartus), allowing their algorithms to map the boolean equations seamlessly into Look-Up Tables (LUTs) on an FPGA architecture, optimizing for both area and propagation delay.

2 Directory: Full_Adder_B

2.1 Architecture: Behavioral Abstraction

Moving up the abstraction hierarchy, this directory implements the exact same 1-bit Full Adder logic, but utilizes Behavioral Modeling. Behavioral Verilog focuses on *what* the circuit does algorithmically, rather than *how* it is physically wired. This paradigm shift is what allows hardware description languages to scale to processors containing billions of transistors.

This design implements a purely combinational procedural block. Inside the block, a case statement is used to directly map the classical truth table of a Full Adder into hardware. The inputs (two operands and a carry-in) are concatenated into a single 3-bit vector acting as a multiplexer selector. The outputs are assigned directly based on the eight possible input states (from decimal 0 to 7).

This method is highly favored in modern RTL design for complex decoders, Arithmetic Logic Units (ALUs), and complex routing logic. It drastically improves human readability, self-documents the truth table within the code, and significantly reduces the likelihood of logical errors during manual boolean minimization (like Karnaugh maps), leaving the heavy lifting of gate-level optimization entirely to the synthesis compiler.

3 Directory: Full_Adder_g

3.1 Architecture: Gate-Level Structural Design

Contrasting the previous abstractions, the Full_Adder_g directory represents the lowest level of digital design: Gate-Level Structural Modeling. This is the closest a digital designer gets to

drawing a physical schematic using code.

Here, the Verilog code acts as a direct netlist. Built-in primitive logic gates (XOR, AND, OR) are explicitly instantiated and connected using internal wires. This directory demonstrates a deep, fundamental understanding of standard CMOS gate topologies and the physical layout of digital circuits. While rarely used for massive modern systems due to severe scalability issues and poor maintainability, structural modeling remains critical in specific niches. For instance, when designing Application-Specific Integrated Circuits (ASICs), an engineer might drop down to gate-level structural code to manually instantiate specific standard cells to achieve absolute, deterministic control over the exact hardware footprint, routing congestion, and critical-path propagation delay.

4 Directory: Counter_behavioral

4.1 Architecture: Synchronous Sequential Logic

This directory marks the transition from combinational logic (where outputs depend solely on current inputs) to sequential logic (where outputs depend on current inputs and past states). It introduces a 2-bit Up/Down Counter designed using behavioral sequential modeling.

Sequential modeling is defined by its sensitivity to clock edges, which forces the synthesis compiler to infer physical memory elements—specifically D-Type Flip-Flops (Registers). The architecture features synchronous and asynchronous reset conditions, ensuring the state machine can always be driven to a safe, known state upon power-up.

Instead of writing boolean logic for the counting mechanism, high-level arithmetic operators (addition and subtraction) are utilized. The compiler intelligently interprets these operators and automatically synthesizes the optimal adder/subtractor logic circuitry to feed the D-inputs of the inferred flip-flops. Furthermore, a multiplexer-like behavior is inferred by conditional routing statements to decide whether the incremented or decremented value is fed back into the state register based on a directional control signal.

5 Directory: Counter_gate

5.1 Architecture: Primitive Sequential State Control

This directory implements the 2-bit Up/Down counter by abandoning high-level addition operators and manually defining the Next-State logic using boolean equations derived from Karnaugh maps.

The exact toggle conditions for the Least Significant Bit (LSB) and Most Significant Bit (MSB) are explicitly mapped out. The logic determines precisely when a bit should flip based on its current state and the directional control pin. This approach bridges structural combinational logic with behavioral sequential memory. It demonstrates an understanding of the underlying mathematics of state machines, showing that an adder is not strictly necessary to build a counter; purely optimized toggle logic is often faster and consumes fewer logic gates.

6 Directory: counter

6.1 Architecture: Pure Structural Instantiation

This directory takes structural design to its ultimate conclusion by completely abandoning procedural blocks for memory.

First, a custom D-Flip-Flop module is explicitly engineered from scratch. Then, the top-level counter structurally instantiates these individual flip-flop modules alongside primitive combinational logic gates. This directory proves mastery over hierarchical modular design in Verilog. It demonstrates how massive, complex systems (like microprocessors) are actually built in reality: by carefully wiring together thousands of smaller, pre-verified sub-components (instances) to form a highly robust Register Transfer Level (RTL) architecture.

7 Directory: Fsm

7.1 Architecture: Finite State Machine Control Logic

The `Fsm` directory contains the cornerstone of digital control theory: a multi-state Finite State Machine. FSMs are the "brains" behind almost every complex digital operation, from simple traffic lights to CPU instruction decoders and network packet routers.

The architecture strictly adheres to the industry-standard three-block methodology, ensuring glitch-free operation and easy debugging:

1. **State Memory (Sequential):** A clocked sequential block that safely transitions the current state to the next state strictly on the rising edge of the system clock.
2. **Next-State Logic (Combinational):** A purely combinational block that evaluates the current state and external inputs to determine the subsequent state mathematically.
3. **Output Logic (Combinational/Registered):** A block dictating the output vectors. This specific design demonstrates a hybrid approach, generating both Moore outputs (dependent strictly on the current state, offering high stability) and Mealy outputs (dependent on state and immediate inputs, offering faster response times).

8 Directory: Lab_5_alu

8.1 Architecture: Parameterizable Datapath Processing

The Arithmetic Logic Unit (ALU) is the computational heart of any processing architecture. This directory implements a highly scalable, robust ALU. A key feature of this design is the use of Verilog parameters, allowing the same exact codebase to synthesize an 8-bit microcontroller datapath, a 32-bit ARM processor datapath, or a 64-bit desktop CPU datapath with zero code modifications.

Controlled by a dedicated multi-bit opcode, the internal combinational logic synthesizes into a wide, complex multiplexer. This multiplexer routes the operands through distinct, parallel hardware functional blocks (adders, bitwise logic gates). Crucially, the architecture includes a zero-detection flag. This flag is evaluated in parallel with the arithmetic operation and is

an absolute necessity for processor architectures, as it enables conditional branching (e.g., "Branch if Equal" or "Jump if Zero") in software execution.

9 Directory: Lab_3_shift

9.1 Architecture: Serial-to-Parallel Interfacing

This directory explores data serialization, a fundamental concept for communication protocols such as SPI, I2C, PCIe, or UART. It integrates a Serial-In Parallel-Out (SIPO) shift register directly with the previously designed ALU.

The architectural premise is that instructions and operands arrive from an external source as a slow, single-wire serial bitstream. The SIPO register clocks this data in sequentially, reconstructing the data. Once a full, wide instruction word is constructed in parallel, it is structurally routed as a bus to the ALU. This precisely simulates how microcontrollers receive data from external serial sensors or memory peripherals and format it for high-speed parallel processing internally.

10 Directory: Lab_7_register

10.1 Architecture: Datapath Storage Elements

This folder contains the design for a standard CPU general-purpose data register. It utilizes a synchronous load enable signal. In physical hardware, this typically translates to a feedback multiplexer placed directly before the input of a D-Flip-Flop. When the load signal is high, the multiplexer routes new data into the flip-flop; when low, it routes the flip-flop's own output back into itself, retaining the state. This parameterizable module serves as the atomic building block for CPU Register Files, pipeline staging registers, and cache memory lines.

11 Directory: Lab_8_memory

11.1 Architecture: Static RAM (SRAM) Modeling

This directory simulates a parameterizable block of Random Access Memory (RAM). In hardware description languages, this is modeled using a 2D array of registers.

The architecture supports synchronous write operations, which are strictly evaluated on the clock edge alongside a write-enable flag. This is crucial for preventing data corruption during the transient periods when address lines are changing. Read operations are handled based on read-enable logic. This SRAM module is essential for simulating Data Memory (for variable storage) and Instruction Memory (for program code storage) in classic Von Neumann or Harvard CPU architectures during the testbench phase.

12 Directory: Lab_9_counter

12.1 Architecture: Program Counter (PC) Mechanics

Designed specifically for integration into a CPU, this counter includes both standard counting functionality and asynchronous/synchronous load (jump) functionalities. In a microprocessor architecture, this serves as the Program Counter (PC). Under normal execution, it increments by one on each cycle to fetch the next sequential instruction from memory. However, when the CPU executes a JUMP, CALL, or BRANCH instruction, the controller asserts the load signal, forcing the counter to abandon its sequence and immediately latch a specific memory address, thereby altering the flow of software execution.

13 Directory: Lab_6_controller

13.1 Architecture: CPU Instruction Decoder and FSM

This directory houses the "Brain" of a custom-designed Central Processing Unit. The Controller utilizes deeply nested combinational logic to decode the incoming instruction opcode into specific execution flags (e.g., ADD, JMP, LDA, STO).

Simultaneously, it tracks the machine cycle phase (Fetch, Decode, Execute). Based on the exact intersection of the current opcode and the current phase of the machine cycle, the controller asserts distinct, perfectly timed control signals (read, write, load_instruction_register, increment_program_counter). This module acts as the grand orchestrator, managing the flow of data across buses and between all other components (Memory, ALU, Registers) to successfully execute software instructions in hardware.

14 Directory: Lab_4_Top_module

14.1 Architecture: Advanced SoC Integration & Data Routing

This directory represents a complex, System-on-Chip (SoC) level integration. It brings together Memory, a Parallel-to-Serial converter (PISO), a Serial-to-Parallel converter (SIPO), and an ALU into a single, cohesive Top Module.

The architectural brilliance here lies in the timing, synchronization, and data path routing. Data is fetched from the RAM block and loaded into a PISO register. The PISO utilizes its own internal mini-FSM to carefully clock the data out bit-by-bit. This serial stream traverses a simulated internal bus to a SIPO, which reconstructs the parallel word and feeds it to the ALU for computation.

This directory demonstrates advanced interface handling, structural wiring of wide buses, handling of valid/enable handshaking signals between asynchronous or semi-synchronous domains, and comprehensive timing validation across multiple interacting digital units.

15 Directory: Final project

15.1 Graduation Project: FPGA-Based Multi-Antenna Controller for Microwave Head Imaging

This directory contains the capstone engineering achievement of the NTI training program. The project focuses on the digital control infrastructure for an advanced medical imaging device. The system acts as the digital nervous system for a diagnostic sensing helmet equipped with a 16-antenna array, designed for non-invasive microwave head imaging (used in detecting strokes, tumors, or internal bleeding).

The scope of this digital design is purely focused on the control logic targeted for a Field Programmable Gate Array (FPGA). The FPGA's role is to orchestrate the precise timing, sequential switching, data acquisition, and PC communication required to drive the external Analog/RF (Radio Frequency) front-end.

15.1.1 The Problem Statement

Microwave head imaging relies on transmitting a specific microwave frequency into the biological tissue and measuring the scattered/reflected signals to construct a 3D dielectric map of the brain. To achieve high-resolution imaging, a helmet is fitted with an array of 16 individual antennas (A1 through A16) strategically positioned around the skull.

The engineering challenge arises in the coordination of these antennas. If all antennas transmit simultaneously, the resulting electromagnetic interference and signal collision would render the data completely useless. Therefore, the system must execute a precise, sequential scan. Every single antenna must be individually:

- Selected and electrically isolated from the others.
- Connected to the RF transceiver via an analog switch matrix.
- Allowed a brief stabilization period (RF settling time).
- Triggered to transmit and receive.
- Sampled by an Analog-to-Digital Converter (ADC).
- Read out and stored in memory before immediately moving to the next antenna.

Using standard microcontrollers (like an Arduino or STM32) for this task can introduce software interrupt latency, jitter, and non-deterministic timing, which corrupts high-frequency RF data.

15.1.2 Our Proposed Solution: FPGA Hardware Control

To overcome the limitations of software-based microcontrollers, our team designed a custom digital controller in Verilog HDL, intended for synthesis on an FPGA.

Unlike software, which executes sequentially line-by-line, an FPGA operates at the pure hardware level. This provides true parallelism and deterministic, nanosecond-level precision. Our solution guarantees that the switching matrices are toggled, the ADC is triggered, and the memory is written to on exact, predictable clock cycles without any operating system overhead.

15.1.3 System Architecture & Module Integration

The system is organized as a layered pipeline connecting the host PC to the analog RF front-end. The FPGA acts as the central 'Microwave Controller', divided into specialized sub-modules to handle the massive data flow:

1. **UART RX & TX (Universal Asynchronous Receiver-Transmitter):** The communication bridge. The RX module listens for commands from the host PC (e.g., START, STOP, RESET). Once the scan is complete, the TX module serializes the stored memory data and streams the final results back to the PC for image processing and rendering.
2. **Antenna Selector:** A synchronous counter responsible for keeping track of the current scan index. It generates a 4-bit select code (from 0000 to 1111) representing the current active antenna.
3. **Switch Controller:** This module acts as the gatekeeper. It takes the 4-bit code from the Antenna Selector and decodes it into a 16-bit "One-Hot" encoded signal. This signal is sent to the external RF Switch Matrix, guaranteeing that the path to the selected antenna is electrically enabled while strictly disabling the paths to the other 15 antennas to prevent interference.
4. **ADC Controller & Memory:** Once the RF signal is received, the ADC controller issues a precise "Start Conversion" trigger to the external Analog-to-Digital Converter. It waits for a "Data Ready" flag, captures the digital sample, and routes it to the correct address in the internal FPGA RAM, ensuring Measurement 1 aligns with Antenna 1, Measurement 2 with Antenna 2, and so forth.

15.1.4 The FSM: Orchestrating the Cycle

The most critical module in the project—and the brain of the entire operation—is the main Finite State Machine (FSM). The FSM ensures that the sequential scanning cycle executes flawlessly.

Upon receiving a START command, the FSM transitions out of the IDLE state and enters a rigorous, self-repeating loop:

- **SELECT_ANTENNA:** Instructs the Antenna Selector to output the current 4-bit index.
- **ENABLE_RF_SWITCH:** Commands the Switch Controller to route the physical RF path.
- **WAIT:** A critical state that pauses the digital system for a predefined number of clock cycles, allowing the physical analog RF signals to settle and stabilize.
- **ADC_START & STORE:** Triggers the external ADC, captures the valid sample, and commits it to the internal Memory block.
- **CHECK_LOOP:** Evaluates if all 16 antennas have been scanned. If not, it increments the index and loops back to SELECT_ANTENNA.
- **DONE & SEND:** Once all 16 antennas are processed, the loop breaks, and the FSM triggers the UART TX module to dump the entire memory contents back to the PC.

15.1.5 Verification and Testing Strategy

Before deploying this logic to physical hardware, the entire digital pipeline was rigorously verified using RTL simulation (ModelSim/Vivado).

The team developed a comprehensive Top-Level Testbench that mocked the external peripherals. The testbench simulates the PC sending a START command via UART, dynamically responds to the FPGA's ADC triggers with mock digital samples, and verifies that the FSM accurately cycles through all 16 states without hanging. By validating the timing diagrams and waveform outputs, the team successfully proved that the digital hardware can reliably and safely manage high-frequency RF peripheral coordination in real-time.

This documentation represents the practical coursework completed during the 90-hour NTI training program. The course was conducted under the instruction of Dr. Ghazal Ahmed Fahmy, Eng. Amr Mostafa, and Eng. Mohamed Salah.